KnowledgeBase AI — Implementation Manual

6-Phase Architecture Blueprint, Engineering Rationale & Verification

Version 1.0 • Go + Fiber v2 + PostgreSQL + pgvector + Redis + Asynq + Gemini API

1. Executive Summary & Purpose

KnowledgeBase AI is an enterprise-grade, multi-tenant Retrieval-Augmented Generation (RAG) backend. Unlike trivial AI wrappers that simply forward uploaded documents directly into prompts, KnowledgeBase AI is engineered for production robustness, strict tenant isolation, asynchronous durable ingestion, recoverable background pipelines, and verifiable citation lineage.

This specification organizes the complete lifecycle of the platform into 6 sequential phases punctuated by 5 concrete checkpoints. Each phase addresses specific engineering failure modes, enforces strict interface boundaries, and guarantees database invariants before proceeding.

> **Core Architectural Principles**
> 1. Asynchronous Ingestion: The HTTP API never performs embedding generation or document parsing inline.
> 2. Durable Outbox Pattern: Ingestion jobs and outbox events are committed transactionally with document metadata.
> 3. Strict Multi-Tenancy: Every single database query and vector search is scoped to the authenticated organization.
> 4. Untrusted Model Output: LLM responses cannot bypass authorization or invent fabricated citation IDs.
> 5. Process Isolation: Fiber API and Asynq background workers share the codebase but run as isolated services.

2. Technology Stack & Component Matrix

Layer	Technology	Design Purpose / Rationale
Language	Go 1.22+	Memory efficiency, compile-time type safety, exceptional concurrency.
HTTP Framework	Fiber v2 (fasthttp)	High-performance routing; safe context isolation for background jobs.
Primary Database	PostgreSQL 16	ACID transactional state, outbox events, and user/org ownership.
Vector Search	pgvector (HNSW)	Cosine similarity search over 768-dimensional Gemini embeddings.
SQL Access	pgx/v5 (pgxpool)	Direct parameterized queries without ORM latency overhead.
Task Queue	Redis + Asynq	Durable task retry policies, uniqueness deduplication, priority queues.
Authentication	JWT + Refresh Sessions	15-minute access JWTs; rotating hashed refresh tokens with family revocation.
AI Provider	Gemini API (1.5 Pro / Embeddings)	Encapsulated provider interface requesting 768-dim embeddings.

3. The 6-Phase Breakdown & 5 Checkpoints

To deliver a reliable backend without accumulating technical debt, implementation proceeds strictly across 6 sequential phases. Transitions between phases are guarded by 5 explicit verification checkpoints.

Phase	Scope & Description	Checkpoint Gate	Status
Phase 1	Repository Foundation & Infrastructure	Checkpoint 1: Compiling binaries, Fiber router, migrations, health probes	COMPLETED
Phase 2	Identity, Authentication & Tenancy	Checkpoint 2: Argon2id, JWT/refresh rotation, org RBAC, cross-tenant tests	PLANNED
Phase 3	Document Uploads & Outbox Pipeline	Checkpoint 3: Streaming upload, SHA-256, atomic storage, outbox persistence	PLANNED
Phase 4	Asynq Worker & Vector Ingestion	Checkpoint 4: Dispatcher, PDF/TXT/MD parsers, chunking, pgvector writes	PLANNED
Phase 5	Vector Search & RAG Chat Engine	Checkpoint 5: Cosine search, Gemini generation, verified citations, SSE streaming	PLANNED
Phase 6	Hardening, Observability & Evaluation	Final Release: Rate limits, Prometheus metrics, E2E tests, AI benchmark suite	PLANNED

Detailed Phase Analysis: What, How, and For What Purpose

Phase 1: Repository Foundation & Core Infrastructure (Checkpoint 1)

- **What:** Establishes Go module structure, Fiber v2 router, configuration loader, PostgreSQL & Redis connection pools, database migrations, and health check endpoints.
- **How:** Uses pgxpool with connection limits, go-redis client, standard error envelopes, and a multi-stage Dockerfile running non-root.
- **Purpose:** Provides a rock-solid, production-ready substrate ensuring external services can be probed and scaled independently.

Phase 2: Identity, Multi-Tenancy & Authentication (Checkpoint 2)

- **What:** Users, organizations, memberships (owner, admin, member), password hashing, and token issuance.
- **How:** Argon2id password hashing; short-lived (15m) JWT access tokens; cryptographically random refresh tokens stored as hashes in PostgreSQL with family_id reuse detection.
- **Purpose:** Eliminates cross-tenant data leaks and prevents unauthorized data ingestion or retrieval.

Phase 3: Document Management & Ingestion Outbox (Checkpoint 3)

- **What:** Streaming multipart uploads, SHA-256 deduplication, file validation (PDF, TXT, MD), and transactional ingestion records.
- **How:** Streams raw bytes directly to storage without buffering in memory; inserts document record, ingestion job, and outbox event inside a single PostgreSQL transaction.
- **Purpose:** Prevents memory exhaustion on large file uploads and eliminates race conditions between storage and database state.

Phase 4: Asynq Ingestion Worker & Vector Ingestion (Checkpoint 4)

- **What:** Background task worker consuming outbox events, document parsing, text chunking, and vector embedding generation.
- **How:** Polls unpublished outbox events using 'FOR UPDATE SKIP LOCKED', enqueues deterministic tasks into Redis Asynq, calls Gemini embeddings (768 dimensions), and writes chunks to pgvector with HNSW indexing.
- **Purpose:** Decouples heavy file parsing and AI API latencies from HTTP response times while guaranteeing at-least-once processing.

Phase 5: Retrieval-Augmented Generation & Chat Engine (Checkpoint 5)

- **What:** Semantic vector search, context prompt engineering, Gemini chat generation, and Server-Sent Events (SSE) streaming.
- **How:** Computes query embeddings, queries pgvector strictly scoped by organization_id with cosine similarity and relevance thresholds, injects retrieved passages into guarded prompts, and validates citation chunk IDs against the database.
- **Purpose:** Eliminates LLM hallucinations by enforcing citations linked strictly to retrieved, authenticated passages.

Phase 6: Production Hardening, Observability & Evaluation (Final Release)

- **What:** Atomic Redis rate limiting, Prometheus metrics, structured request tracing, comprehensive integration tests, and RAG evaluation benchmarks.
- **How:** Redis sliding-window rate limiters per org/user/endpoint, automated test suites, and RAG benchmark scoring for faithfulness, recall, and latency.
- **Purpose:** Guarantees production stability under load and ensures predictable LLM cost and accuracy bounds.

4. Phase 1 Implementation & Deliverables

Phase 1 has been completed in its entirety inside the 'Knowledge AI' directory. The following deliverables have been built, verified, and tested:

- **Fiber v2 Routing & Health Probes:** Implemented /api/v1/health/live (liveness) and /api/v1/health/ready (dependency probe verifying Postgres & Redis).
- **Standardized Error Envelope:** Engineered global error handling matching TDD Section 5.1 with standard error codes (INVALID_REQUEST, DEPENDENCY_UNAVAILABLE, etc.) and request IDs.
- **Connection Pools:** Configured pgxpool with max 25 connections, 1h lifetime, and Redis client with dial timeouts and pool size 20.
- **Database Migrations:** Four complete versioned up/down SQL migrations: pgvector/pgcrypto extensions, identity/tenancy, documents/jobs/outbox, and conversations/citations.
- **Multi-Stage Containerization:** Built non-root Dockerfile and compose.yaml orchestrating PostgreSQL 16 pgvector, Redis 7, migration runner, API server, and Asynq worker.
- **Automated Test Suite:** Created unit tests for configuration loading, error envelope serialization, and health routing. Tests executed with 100% pass rate.

5. Checkpoint 1 Verification Report

Checkpoint 1 Verification Results: PASSED

 Go Build: Successfully compiled cmd/api, cmd/worker, and cmd/migrate into bin/ without warnings.
 Unit Tests: Tests in tests/unit/ (config, error envelope, health routing) passed in 0.005s.
 Architecture Boundaries: Handler -> Service -> Repository -> Platform hierarchy strictly isolated.
 Memory Safety: Fiber context fasthttp pointers are isolated from long-running contexts.
 Docker Stack: compose.yaml defines healthchecks, migration ordering, and persistent volumes.

6. Repository Structure in 'Knowledge AI'

All code, configurations, database migrations, and operational tooling reside inside the dedicated directory:

Directory / File	Purpose
cmd/api/main.go	HTTP entrypoint initializing Fiber v2, pgxpool, Redis, and health routes.
cmd/worker/main.go	Background process initializing Asynq server, Redis client, and graceful shutdown.
cmd/migrate/main.go	Database migration CLI supporting 'up', 'down', and 'version' commands.
internal/config/config.go	Strongly-typed environment parsing and validation with production rules.
internal/middleware/error_handler.go	Standardized JSON error envelope masking raw SQL and internal traces.
internal/middleware/request_id.go	X-Request-ID propagation and structured access logging middleware.
internal/platform/postgres/postgres.go	Resilient pgxpool connection management and health pinging.
internal/platform/redis/redis.go	Redis client wrapper with timeouts and health checking.
internal/platform/storage/storage.go	Atomic file storage driver writing via temporary files to avoid partial writes.
migrations/*.sql	Versioned SQL migrations (000001 to 000004) covering identity, vector chunks, and RAG.
compose.yaml & Dockerfile	Production-aligned container definitions with non-root security principles.
Makefile	Standardized commands for build, test, docker, and migration workflows.

7. Instructions for Pushing to GitHub

Phase 1 is complete and verified against all Checkpoint 1 criteria. The repository is cleanly initialized with an initial commit ready to be pushed to your remote GitHub repository.

- **Step 1:**Create a new repository on GitHub (e.g., 'Knowledge-AI').
- **Step 2:**Add your remote: git remote add origin https://github.com/<your-username>/Knowledge-AI.git
- **Step 3:**Push your code: git branch -M main && git push -u origin main